

Objetivo

Esta apostila mostra como utilizar o **Prisma v4** com **PostgreSQL**, acessando um banco populado (`public.escola`) por meio de **RAW SQL**.

Passo 1 — Instalar Prisma v4

```
npm i -D prisma@4
npm i @prisma/client@4
```

Passo 2 — Inicializar o Prisma

```
npx prisma init --datasource-provider postgresql
```

Edite o arquivo `.env` e configure a URL de conexão:

```
DATABASE_URL="postgresql://usuario:senha@localhost:5432/seubanco?
schema=public"
```

Passo 3 — Carregar um banco existente

Como o banco já existe e está populado, rode:

```
npx prisma db pull
```

O Prisma lerá a estrutura e criará o arquivo `prisma/schema.prisma`.

Mesmo que você use **RAW SQL**, manter esse schema garante compatibilidade e tipagem.

Passo 4 — Gerar o cliente Prisma

```
npx prisma generate
```

Isso permite importar e usar o cliente:

```
import { PrismaClient } from "@prisma/client"
const prisma = new PrismaClient()
```

Arquitetura recomendada

```
escolas/
├─ prisma/
│   └─ schema.prisma
├─ src/
│   ├─ server.ts
│   ├─ prisma.ts
│   ├─ entities/
│   │   └─ IEscola.ts
│   ├─ rotas/
│   │   └─ escola.routes.ts
│   ├─ repositories/
│   │   └─ escola.repository.ts
│   └─ controllers/
│       └─ escola.controller.ts
├─ .env
├─ tsconfig.json
└─ package.json
```

src/prisma.ts

```
import { PrismaClient } from "@prisma/client"
export const prisma = new PrismaClient()
```

Interface IEscola (src/entities/IEscola.ts)

Usaremos apenas os campos mais relevantes, conforme solicitado:

```
export interface IEscola {
  id_escola: number
  codigo_mec: number | null
  nome: string | null
  data_fundacao: Date | string | null
  email: string | null
  numero: string | null
  complemento: string | null
  bairro: string | null
  cep: string | null
  municipio: string | null
  data_cadastro: Date | string
}
```

Repository

(src/repositories/escola.repository.ts)

As consultas serão realizadas com **RAW SQL** parametrizado usando `prisma.$queryRaw` e `prisma.$executeRaw`.

Explicação dos termos novos e funcionalidade:

- `$queryRaw` : Retorna dados. Usado quando a query retornará valores da tabela. Uso principalmente com SELECT.
- `$executeRaw` : Não retorna dados. Usado quando a query retornará apenas as linhas afetadas. Uso principalmente com INSERT, UPDATE e DELETE.
- `<IEscola[]>` : Tipagem dos dados que serão capturados e enviados.
- `Partial<IEscola>` : Permitir criar um objeto **com apenas parte** das propriedades.

```
import { prisma } from "../prisma"
import type { IEscola } from "../entities/IEscola.js"

export class EscolaRepository {
  constructor(private db = prisma) {}

  async listar(limit = 50, offset = 0): Promise<IEscola[]> {
    const rows = await this.db.$queryRaw<IEscola[]>`
      SELECT id_escola, codigo_mec, nome, data_fundacao, email,
             numero, complemento, bairro, cep, municipio, data_cadastro
      FROM public.escola
      ORDER BY id_escola`
  }
}
```

```

        LIMIT ${limit} OFFSET ${offset};
    `
    return rows
}

async buscarPorId(id: number): Promise<IEscola | null> {
    const rows = await this.db.$queryRaw<IEscola[]>`
        SELECT id_escola, codigo_mec, nome, data_fundacao, email,
               numero, complemento, bairro, cep, municipio, data_cadastro
        FROM public.escola
        WHERE id_escola = ${id};
    `
    return rows[0] ?? null
}

async criar(data: Partial<IEscola>): Promise<IEscola> {
    const rows = await this.db.$queryRaw<IEscola[]>`
        INSERT INTO public.escola
        (codigo_mec, nome, data_fundacao, email,
         numero, complemento, bairro, cep, municipio, data_cadastro)
        VALUES
        (${data.codigo_mec}, ${data.nome}, ${data.data_fundacao},
         ${data.email}, ${data.numero}, ${data.complemento},
         ${data.bairro}, ${data.cep}, ${data.municipio}, NOW())
        RETURNING id_escola, codigo_mec, nome, data_fundacao, email,
                  numero, complemento, bairro, cep, municipio, data_cadastro;
    `
    return rows[0]
}

async atualizar(id: number, data: Partial<IEscola>): Promise<IEscola> {
    const rows = await this.db.$queryRaw<IEscola[]>`
        UPDATE public.escola
        SET
            codigo_mec = COALESCE(${data.codigo_mec}, codigo_mec),
            nome = COALESCE(${data.nome}, nome),
            data_fundacao = COALESCE(${data.data_fundacao}, data_fundacao),
            email = COALESCE(${data.email}, email),
            numero = COALESCE(${data.numero}, numero),
            complemento = COALESCE(${data.complemento}, complemento),
            bairro = COALESCE(${data.bairro}, bairro),
            cep = COALESCE(${data.cep}, cep),
            municipio = COALESCE(${data.municipio}, municipio)
        WHERE id_escola = ${id}
        RETURNING id_escola, codigo_mec, nome, data_fundacao, email,
                  numero, complemento, bairro, cep, municipio, data_cadastro;
    `
}

```

```

    \
    return rows[0]
}

async deletar(id: number): Promise<void> {
    await this.db.$executeRaw`
        DELETE FROM public.escola WHERE id_escola = ${id};
    `
}
}

```

Controller

(src/controllers/escola.controller.ts)

```

// src/controllers/escola.controller.ts
import { Request, Response } from "express"
import { EscolaRepository } from "../repositories/escola.repository.js"

export class EscolaController {
    private repo: EscolaRepository

    constructor() {
        this.repo = new EscolaRepository()
    }

    // GET /escolas
    listar = async (req: Request, res: Response) => {
        const limit = Number(req.query.limit ?? 50)
        const offset = Number(req.query.offset ?? 0)
        const escolas = await this.repo.listar(limit, offset)
        res.json(escolas)
    }

    // GET /escolas/:id
    obter = async (req: Request, res: Response) => {
        const id = Number(req.params.id)
        const escola = await this.repo.buscarPorId(id)
        if (!escola) return res.status(404).json({ erro: "Escola não encontrada"
    })
        res.json(escola)
    }
}

```

```
// POST /escolas
criar = async (req: Request, res: Response) => {
  const nova = await this.repo.criar(req.body)
  res.status(201).json(nova)
}

// PUT /escolas/:id
atualizar = async (req: Request, res: Response) => {
  const id = Number(req.params.id)
  const att = await this.repo.atualizar(id, req.body)
  res.json(att)
}

// DELETE /escolas/:id
deletar = async (req: Request, res: Response) => {
  const id = Number(req.params.id)
  await this.repo.deletar(id)
  res.status(204).send()
}
}
```

Rotas Escola (src/rotas/escola.routes.ts)

```
import { Router } from "express"
import { EscolaController } from "../controllers/EscolaController"

const controller = new EscolaController()

export const escolaRoutes = Router()

escolaRoutes.get("/escolas", controller.listar)
escolaRoutes.get("/escolas/:id", controller.obter)
escolaRoutes.post("/escolas", controller.criar)
escolaRoutes.put("/escolas/:id", controller.atualizar)
escolaRoutes.delete("/escolas/:id", controller.deletar)
```

Servidor Express (src/server.ts)

```
import express from "express"
import { EscolaController } from "../controllers/escola.controller"
import { escolaRoutes } from "../rotas/escola.routes"

const app = express()
app.use(express.json())

const escolaController = new EscolaController()

app.use("/api", escolaRoutes)

const PORT = process.env.PORT || 3000
app.listen(PORT, () => console.log(`Servidor rodando em
http://localhost:${PORT}`))
```
